



PDF Download
3584666.pdf
02 January 2026
Total Citations: 24
Total Downloads: 1372

 Latest updates: <https://dl.acm.org/doi/10.1145/3584666>

RESEARCH-ARTICLE

Generative Adversarial Networks for Cyber Threat Hunting in Ethereum Blockchain

ELNAZ RABIEINEJAD

ABBAS YAZDINEJAD

REZA M PARIZI

ALI DEGHANTANHA

Published: 08 June 2023
Online AM: 24 February 2023
Accepted: 23 October 2022
Revised: 30 September 2022
Received: 10 December 2021

[Citation in BibTeX format](#)

Generative Adversarial Networks for Cyber Threat Hunting in Ethereum Blockchain

ELNAZ RABIEINEJAD and ABBAS YAZDINEJAD, Cyber Science Lab, School of Computer Science, University of Guelph

REZA M. PARIZI, Decentralized Science Lab, Kennesaw State University

ALI DEGHANTANHA, Cyber Science Lab, School of Computer Science, University of Guelph

Ethereum blockchain has shown great potential in providing the next generation of the decentralized platform beyond crypto payments. Recently, it has attracted researchers and industry players to experiment with developing various Web3 applications for the Internet of Things (IoT), Defi, Metaverse, and many more. Although Ethereum provides a secure platform for developing decentralized applications, it is not immune to security risks and has been a victim of numerous cyber attacks. Adversarial attacks are a new cyber threat to systems that have been rising. Adversarial attacks can disrupt and exploit decentralized applications running on the Ethereum platform by creating fake accounts and transactions. Detecting adversarial attacks is challenging because the fake materials (e.g., accounts and transactions) as malicious payloads are similar to benign data. This article proposes a model using Generative Adversarial Networks (GAN) and Deep Recurrent Neural Networks (RNN) for cyber threat hunting in the Ethereum blockchain. Firstly, we employ GAN to generate fake transactions using genuine Ethereum transactions as the first phase of the proposed model. Then in the second phase, we utilize bi-directional Long Short-Term Memory (LSTM) to identify adversarial transactions in a hunting exercise. The results of the first phase evaluation show that the GAN can generate transactions identical to the actual Ethereum transactions with an accuracy of 82.51%. Also, the results of the second phase show 99.98% accuracy in identifying adversarial transactions.

CCS Concepts: • **Security and privacy** → **Distributed systems security**;

Additional Key Words and Phrases: Ethereum blockchain, threat hunting, GAN, bi-directional LSTM, security, attack detection

ACM Reference format:

Elnaz Rabieinejad, Abbas Yazdinejad, Reza M. Parizi, and Ali Dehghantanha. 2023. Generative Adversarial Networks for Cyber Threat Hunting in Ethereum Blockchain. *Distrib. Ledger Technol.* 2, 2, Article 9 (June 2023), 19 pages. <https://doi.org/10.1145/3584666>

1 INTRODUCTION

In recent years, blockchain has attracted tremendous attention due to its characterized properties such as confidentiality, anonymity, and non-reputability to enhance the security, reliability, and transparency of systems [1]. Blockchain is a secure and immutable ledger of blocks used to store and share data in a distributed manner.

Authors' addresses: E. Rabieinejad, A. Yazdinejad, and A. Dehghantanha, Cyber Science Lab, School of Computer Science, University of Guelph, 50 Stone Rd E, Guelph, ON N1G 2W1, Ontario, Canada; emails: erabiein@uoguelph.ca, ayazdine@uoguelph.ca, adehghan@uoguelph.ca; R. M. Parizi, Decentralized Science Lab, Kennesaw State University, GA, USA; email: rparizi1@kennesaw.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

2769-6472/2023/06-ART9 \$15.00

<https://doi.org/10.1145/3584666>

Blockchain offers a mechanism for people (or entities) who do not know or trust each other to trade or share asset ownership verifiably [2]. Ethereum, as an open blockchain, allows decentralized applications to run on its platform employing smart contracts. Among the many uses of Ethereum, electronic voting mechanisms [3], secure sharing of medical data [4], supply chain and logistics monitoring [5], and **Internet of Things (IoT)** [6] are some of the recent applications. However, despite the growth in the application of Ethereum, reports indicate that it is exposed to many attacks, cyber threats, and malicious activities, which have caused a lot of financial losses [7, 8]. Hackers constantly find new ways to exploit vulnerabilities of decentralized programs on the Ethereum platform for nefarious purposes or launching cyberattacks [9, 10].

Adversarial attacks are another cyber threat category that has been rising in recent years. The goal of adversarial attacks is to generate adversarial data (i.e., inputs) that are almost indistinguishable from natural data, yet defense mechanisms cannot detect them [11, 12]. There is no exception for Decentralized apps (Dapps) that run on the Ethereum platform, and these attacks can victimize them. Hackers can infiltrate Dapps and the Ethereum platform to make losses by generating fake transactions [13] or creating some fake accounts in peer-to-peer networks [14], which are identical to real ones. Indistinguishable transactions in Ethereum generally refer to transactions in which all items are the same as the original transactions and classified as normal in the **intrusion detection system (IDS)**. In fact, by hacking into their IDS and understanding their vulnerabilities, hackers can figure out how to generate a transaction that passes the **machine learning (ML)**-based intrusion detection barrier. These types of exposures are unknown to the system and its developers. These attacks can reduce the value of the cryptocurrency, lose mining rewards, and delay legal transactions. For instance, in 2017, with the increasing number of fake transactions in the memory pool, the verification process was delayed, and increased the mining fee [15]. In another example, hackers using fake ripple transactions stole USD 460 million [16].

ML is one of the most popular methods for threat hunting. Owing to the efficiency and accuracy of ML, threat hunting using this has become mainstream these days [17–19]. In this method, algorithms can quickly process large volumes of data and extract patterns from a train set to identify cyber attacks. Due to this, hackers try to use adversarial attacks to fool ML-based detection models by poisoning the training data or generating fraud samples (identical to normal ones) to confuse the model. So, the model wrongly classifies the data and cannot identify attacks correctly. To make adversarial attacks in blockchain, hackers tend to avoid well-known attack patterns, so an ML-based IDS cannot distinguish between them and a normal blockchain transaction. These ML-based IDSs may perform well in the test environment, but they cannot detect unknown and zero-day attacks in the real world [20]. Hackers monitor and collect validated transactions to make an adversarial attack on the blockchain. Then, hackers use these collected transactions to find the signature of transactions detected as an attack by an ML-based IDS. As a result, they avoid this pattern to fool the ML-based IDS.

The **Generative Adversarial Network (GAN)** is an advanced type of ML learning that has shown good performance in threat hunting [17]. Compared with traditional data augmentation methods, the GAN-based generating method can produce data in the same range and distribution, identical to real ones; however, those methods usually increase datasets' dimensions [21]. GAN capabilities to generate any data distribution similar to the original greatly influence adversarial attacks hunting [22, 23]. By generating adversarial attack patterns, GAN allows ML models to learn from generated data and improve their threat hunting performance [11, 24–26].

This article proposes a GAN-based threat hunting model in the Ethereum blockchain. This ML model consists of two phases. The first phase uses conditional GAN to generate fake transactions for the Ethereum blockchain. The GAN structure consists of a generator network that uses a noise vector and the label of each column in the dataset to create adversarial transactions, and a discriminator network tries to detect fake transactions. The interaction between the generator and discriminator improves both networks' performance and, as a result, generates better adversarial transactions. We then combine the fake transactions with the real ones and create a database containing both types of these transactions. These transactions are vital in our proposed detection model because we use them for training the deep learning model and making it resistant to adversarial attacks. This blend of genuine and fake transactions consolidates a deep learning model pattern, making it difficult for

intruders to fool it. In the second phase, we fed this new dataset to a bi-directional **Long Short-Term Memory (LSTM)** to detect fake transactions and, more precisely, the adversarial attacks in the Ethereum blockchain. The importance of the GAN-based threat hunting model is to prevent successful hacker attacks by disabling the **Cyber Kill Chain (CKC)**. CKC consists of stages: reconnaissance, weaponization, delivery, exploitation, installation, command and control, and action on objectives. Because the normal and fake Ethereum transactions have trained the GAN-based threat hunting model, it can detect adversarial attacks on Ethereum transactions in the delivery stage. Furthermore, our proposed model disturbs the exploitation, installation, command and control, and action on objectives stages. To summarize, the main contributions of this article are as follows:

- A GAN-based threat hunting model is designed and implemented in the Ethereum blockchain. This model employs a customized conditional GAN to create a threat hunting model to avoid adversarial attacks.
- We generate adversarial Ethereum transactions based on Ethereum tabular data using conditional GAN to fool ML-based intrusion detection models.
- We use bi-directional LSTMs to detect adversarial attacks on Ethereum's blockchain by GANs.

Following is an overview of the remainder of the article. First, a review of related work applying GAN and ML for attack and anomaly detection is given in Section 2. Then, the proposed GAN-based model is presented in Section 3. The evaluation results of the proposed model are presented in Section 4, and Section 5 is dedicated to future works. Lastly, Section 6 concludes the article.

2 RELATED WORK

Cyber threat hunting has been well studied and has always been a favorite topic for researchers [17, 27]. ML is a suitable tool to detect attacks [28]. However, limited attack data is a challenge to adopting ML. This section presents related works that used ML and GAN for anomaly and attack adversarial transactions.

Fraud transactions are one of the significant concerns in online e-commerce platforms. For improving a deep learning-based attack detection system for detecting fraud transactions, authors in [29] proposed two attack detection algorithms. These algorithms, named **Augmented Iterative Search (AIS)** and **Rounded Relaxation with Reparameterization (R3)**, were evaluated on TaoBao, one of the largest e-commerce platforms in the world. The result shows that the vulnerability of fraud transactions decreased by nearly 90%. In another work, Authors [30] explore different scenarios for adversarial attacks and defenses on financial transaction records data. Their result shows that generated financial transactions can easily fool the deep learning vulnerable detection model. Then, to enhance the model's robustness, they trained the deep learning model using adversarial transactions. Some works adopt GAN for detection transactions. For instance, Hung Ba [31] designed a new model to detect adversarial credit card transactions using GAN. The role of GAN in this article is to generate artificial data to assist in classifying fraudulent credit card transactions. The author evaluated his model by **Wasserstein-GAN (WGAN)** and **conditional GANs (CGANS)** versions. The results show that WGAN is more stable in training and produces more realistic fraudulent transactions than CGAN. In [13], the authors proposed a conditional GAN-based model to produce **Low-rate Distributed DoS (LDDoS)**. At first, in this model, they modeled the LDDoS attacks feature and fed them to LSTM-based conditional GAN. In conditional GAN, both generator and discriminator adopt two-layer drop-based LSTM. The result shows that the generated data can sometimes cheat the ML model. In another research [11], the authors propose a method for detecting anomalies using the high-level granularity of system logs. Due to this purpose, they adopt an LSTM-based GAN for anomaly detection using system logs. This system, called LogGAN, extracts timestamps, signatures, and attributes from system logs and then feeds them to the GAN.

In GAN, both the generator and discriminator use LSTM. The results show that this model increases the efficiency of anomaly detection compared to the model due to using the GAN and solving imbalanced data. In [12], Zhi Chen and his colleague proposed an anomaly detection method using GAN and ensemble learning. This model consists of a generator and multiple discriminators. The discriminator includes two classifiers. The

first one is responsible for detecting the real and fake data. Another one is responsible for detecting normal and abnormal data. Using multiple discriminators solves the bias classification problem. In [32] E. Seo et al. provide an anomaly detection method using GAN for in-vehicle networks and Controller Area networks. GAN was adopted to resolve the limited known attack data challenge for in-vehicle networks. The evaluation of four types of attack: DoS, fuzzy, RPM, and Gear, shows the model's significant performance, which is 100% approximately.

Moreover, the authors in [33] provide an unsupervised anomaly detection system using GAN for cyber-physical complex systems associated with many sensors and actuators. LSTM with three hidden layers was used in generator and discriminator networks to analyze the time series data and detect anomalies. In another research article [34], the authors modified the original GAN loss function. The new model focuses on distributing generated data on actual data boundaries instead of overlapping them. The model was tested on MNIST, CIFAR10, and KDD99 datasets, and the result shows an improvement in anomaly detection performance. In another work [35], authors acknowledge that sIDSs usually have many **false alarm rates (FARs)** because of the lack of enough prior knowledge of malicious samples. Based on the authors' philosophy, adding some malicious samples improves the robustness of the model against FARs. In this way, they proposed an IDS using GAN, which can add some malicious samples to the model during training. Their approach can help the model to be trained more realistically and decrease the FARs of the model. In addition, authors in [36] proposed a blockchain and GAN-based IDS for IoT. Their proposed model contained three levels. The first level is dedicated to collecting IoT data from smart objects and recording them in the blockchain. Then, feature selection and feature mapping have been made in level 2, and finally, data classification into normal and abnormal is done in level 3. Their experimental result shows that their proposed model performs better than existing methods.

3 THE PROPOSED MODEL

Today, blockchains face various attacks. Hackers try to attack blockchain-based systems by exploring vulnerabilities in these systems. Recently, ML-based IDSs have been used to detect potential attacks in blockchains [37]. These ML-based IDSs require massive, labeled data and malware properties to consider optimal features for detecting abnormal transactions [38]. In addition, recently, ML-based IDSs have faced adversarial attacks and unknown attacks that can fool them [39]. These attacks avoid well-known attack patterns. So an ML-based IDS cannot find a malicious pattern and distinguish between abnormal and normal transactions. These ML-based IDSs may perform well in the test environment, but they cannot detect unknown, and zero-day attacks in the real world [20]. GAN can generate fraudulent transactions from genuine transactions and makes an opportunity for the IDS to be trained with many fake transactions and unknown attacks. Furthermore, adopting GAN allows ML-based IDSs to act better in the real world [40].

For producing fraudulent transactions, hackers monitor and collect validated transactions. Then, they analyze these collected transactions to find the pattern and signature of transactions that can be detected as an attack by an ML-based IDS and, as a result, avoid this pattern to fool the ML-based IDS. These adversarial transactions have the potential to mislead the ML-based detection systems. Our proposed model adopts the GAN to simulate these attacks and trains a bi-directional LSTM to be prepared for detecting them. Compared with traditional data augmentation methods, the GAN-based generating method has characteristics that make it suitable for attack detection in the blockchain. In the first step, we needed to generate transactions entirely identical to the original transactions. Adopting the GAN-based generating method for generating transactions has several advantages. First of all, the GAN-based generating method has shown better performance for generating and distributing data in a restricted range. Moreover, the GAN is ideal for complex environments with high dimensions. In our model, blockchain makes systems more complex and generates many transactions with high dimensions. Furthermore, these compelling reasons led us to use GAN for generating fake transactions [41].

Figure 1 represents the proposed model architecture. At first, we will use a conditional GAN to generate adversarial transactions in Ethereum. This model consists of two **Deep Neural Networks (DNN)**, a generator, and a discriminator. The generator part of a GAN learns to create fake data by incorporating feedback from the

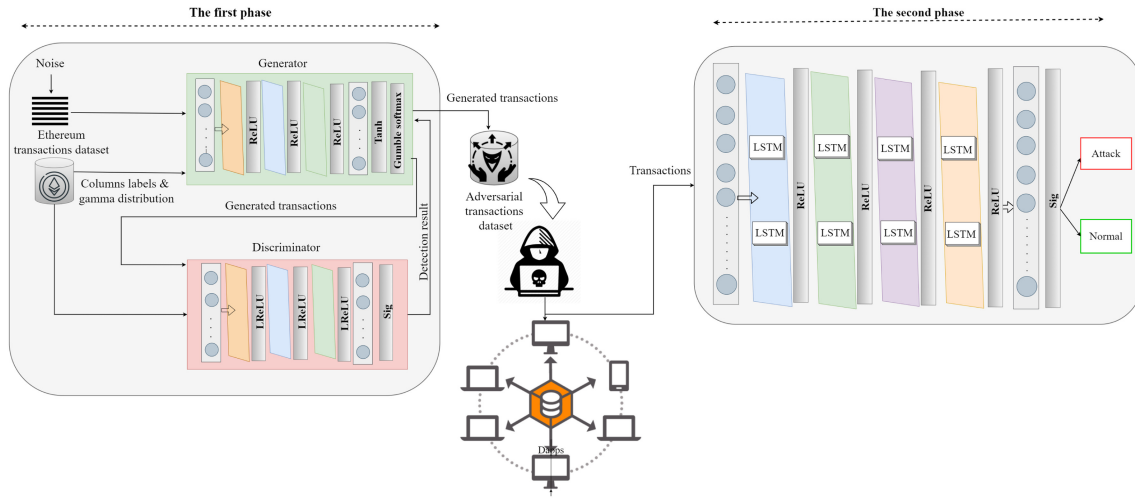


Fig. 1. The proposed model for GAN-based threat hunting in Ethereum blockchain.

discriminator. It learns to make the discriminator classify its output as real. On the other hand, the discriminator is simply a classifier. It tries to distinguish real data from the data created by the generator. Generators use noise and the label of each column to generate data and use discriminator feedback to improve the similarity of generated data to real ones. Because the discriminator compares the generated data with the real data and then gives feedback to the generator, it effectively improves generator performance. The exact definition of the generator performance is the production of transactions similar to normal Ethereum ones, and the more similar these transactions are generated, the higher the performance. In the second phase, we combine the generated adversarial transactions with the real transactions and feed them into the threat hunting model. Our goal is to identify the adversarial transactions with the highest possible accuracy. For this purpose, we use a bi-directional LSTM model with four hidden layers connected using two LSTM units.

3.1 The First Phase: Adversarial Transaction Generation by GAN

To generate adversarial transactions, we need a dataset containing real Ethereum transactions. Most datasets related to Ethereum transactions are tabular, and the regular version of GAN does not work on tabular datasets for reasons such as mixed data types, imbalanced categorical columns, non-gaussian distributions, and so on. To overcome this issue, we use a version of the GAN developed by MIT, called CTGAN [42]. CTGAN is a conditional GAN which supports tabular data. The GAN structure consists of neural networks called generators and discriminators. Both of generator and discriminator use a feedforward, fully connected neural network. The reason for using a feedforward, fully connected DNN is that the GAN adopts two DNNs, and its training process is time and energy-consuming. Furthermore, adopting complicated ML models for the generator and discriminator increases this training time and energy [43].

The generator network uses a noise vector to generate data and the label of each column, which indicates the columns' types, as input. The process of adversarial transaction generation by GAN has been shown in Algorithm 1. In *Initialization* phase, the number of adversarial transactions to be generated and the original Ethereum dataset with n transactions have been fed to the model. At first, the generator randomly selects the column D_j from all columns $D = [1, \dots, n]$ in the dataset to produce its data. The generator gets each column label (v^*) and gamma distribution (k^*) to generate data correctly. Then concatenating these columns is a label with their gamma distribution (c). Finally, the gamma distribution is used to get the data distribution range. The gamma distribution indicates positive numbers, and because transactions' data in each column is positive, this

function is suitable for use. Due to this distribution function, the generator produces data in a positive range, and there is less distance between generated and actual transactions. Therefore, less time is needed to improve generator performance and produce more identical transactions. To calculate gamma distribution, according to Equation (1), for data in each column, data shapes, scales, and locations are received, and the gamma function for that column is calculated based on these parameters. In this equation, y is a shape parameter, k represents the scale parameter, b is the location parameter, and indicates the gamma function.

$$f(x) = \frac{(\frac{x-b}{k})^{y-1} \exp(-\frac{x-b}{k})}{k\Gamma(y)}. \quad (1)$$

A noise vector (z) is used as a generator neural network input in the generation step. As we can see in the algorithm, the input layer concatenates the input vector with the noise vector and enters the hidden layer ($G_{DNN}(z, c^*)$). We adopt three hidden layers in the generator, each with 256 neurons. Increasing the number of layers to three will cause the model to slow down, while its accuracy will not differ significantly. Therefore, we tried to select the number of layers and the number of neurons with the best performance according to execution time and accuracy. We use the **Rectified Linear Activation Unit (ReLU)** as an activation function. This function allows faster and more effective training of deep neural architectures on large and complex datasets, and in addition, in a randomly initialized network, only about 50% of hidden units are activated (have a non-zero output). It should be mentioned that because ReLU propagates data in just one direction (positive), this function has fewer vanishing gradient problems compared to sigmoidal activation functions. Equation (2) shows the ReLU formula, where x indicates the input to a neuron. It avoids and rectifies the vanishing gradient problem.

$$ReLU(x) = \text{Max}(0, x) = \begin{cases} x, & x < 0 \\ 0, & x \geq 0 \end{cases}. \quad (2)$$

The three hidden layers process data and then send it to declare and display the output value to the last layer. The final layer is responsible for declaring the amount of data received and its kind of data. In Ethereum transactions, the values of some fields are categorical, such as input, and some are continuous, such as value. Given that we have both discrete and continuous data, we use two types of activator functions. We use the tanh activation function for continuous values, which transforms data to the range of -1 to 1 . This function can cover the negative values that existed in Ethereum transactions. This activation function is described in Equation (3), and in this equation, x indicates observations.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (3)$$

The activation function used for discrete columns is gumbel softmax [44]. Gumbel softmax is a continuous distribution that can approximate samples from a categorical distribution. Thus, the output of this mechanism is a continuous variable rather than a quantized representation. Gumbel softmax is shown in Equation (4), where x presents a discrete variable with the class probability of $p_1, p_2, \dots, p_k$, y represents the softmax temperature and $g_1, g_2, \dots, g_k$ show on Gumbel samples.

$$xi = \frac{\exp(\frac{\log p_i + g_i}{y})}{\sum_{j=1}^k \exp(\frac{\log p_j + g_j}{y})} \text{ for } i = 1, \dots, k. \quad (4)$$

These activator functions help the output layer to generate data more accurately according to the original transactions. The generated transactions are the input of the discriminator network.

The generated and actual transactions are combined in the first layer and enter the hidden layers. There are three hidden layers with 256 neurons. We use **Leaky ReLU (LReLU)** as an activation function that is represented



Fig. 2. The CKC structure.

in Equation (5). Because ReLU immediately transforms the negative values to zero and may not fit well on the data, LReLU is used.

$$LReLU(x) = \text{Max}(0.1x, x). \quad (5)$$

The hidden layers in the discriminator neural network try to distinguish the fake type by examining the transactions and finally sending their processing to the output layer. The output layer, which uses the Sigmoid (Sig) activation function, declares the result of its detection. Equation (6) represents this function. The Sig function is used primarily for models where we have to predict the probability as an output, and the likelihood of anything is between 0,1. Since the likelihood of detecting attacks is between 0 and 1, Sig is the right choice. In this equation:

$$\text{Sig}(x) = 1/(1 + e^{-x}). \quad (6)$$

We consider the **Wasserstein Generative Adversarial Network (WGAN)** [45] cost function and Adam optimizer for stochastic gradient descent in the training process. WGAN is an extension to the GAN that improves the stability when training the model and provides a loss function that correlates with the quality of generated data. We have two loss functions for these two DNNs. The loss function of the discriminator leads it to detect fake transactions better each time. In contrast, the generator's loss helps it improve its performance to generate better fake transactions. The lost functions for the discriminator and generator are shown in Equations (7) and (8), respectively.

$$\text{Discriminator_loss} = W1/m \sum_{i=1}^m [f(x_i) - f(G(z_i))], \quad (7)$$

$$\text{Generator_loss} = G1/m \sum_{i=1}^m f(G(z_i)). \quad (8)$$

In these equations, W and G indicate the discriminator and generator initial parameters, and m shows batch size. Moreover, f indicates the value of 1-Lipschitz function, that is according to Equation (9) in which C is a constant value.

$$|f(x_1) - f(x_2)| \leq C|x_1 - x_2|. \quad (9)$$

The adversarial transactions combine with the normal transactions to form a new dataset used in the second phase.

3.2 The Second Phase: Ethereum Adversarial Attack Detection Using Deep Recurrent Neural Network

Adversarial attacks usually do not have a unique signature and pattern. They can furthermore hide easily from cyber attack hunting models. In our proposed model, due to generating adversarial attacks pattern in the first phase, our threat hunting model can detect fake transactions quickly and make it hard for attackers to penetrate the system. It is crucial to swiftly detect adversary transactions to prevent high amounts of damage and loss in the system. For more illustration, we consider the process of the adversarial attack as a CKC, according to Figure 2 [46]. The CKC breaks down the attacks' process into consecutive stages: reconnaissance, weaponization,

ALGORITHM 1: Generating adversarial transaction for Ethereum blockchain

```

1  Input :  $E = [x_0, \dots, x_n] \Rightarrow$  an Ethereum dataset with n transactions;
2  Output : generated transactions;
3  Initialization;
4   $S \Rightarrow$  Number of adversarial transactions to be generated
5   $i \& j = 1$ 
6  while ( $i \leq S$ ) do
7    while ( $j \leq n$ ) do
8      Generator selects a column  $D_j$  from all columns  $D=[1, \dots, n]$ ;
9      Use one-hot encoder to convert the column label to corresponded vector  $\Rightarrow v^*$ ;
10     Calculate gamma distribution for the value of  $D \Rightarrow k^*$ ;
11      $v^* \otimes k^* \Rightarrow c^*$ ;
12      $i=i+1$ ;
13   end
14 end
15 while (number of iteration) do
16   Initialization generator network;
17   input: epochs, batch, lr(learning rate)
18    $Z =$  the noise vector
19   Generator =  $G_{DNN}(z, c^*)$ 
20   while (train samples) do
21     Generator.fit(samples, epochs, batch);
22     Apply  $\Rightarrow$  Adam as optimizer;
23     Apply  $\Rightarrow \frac{G}{m} \sum_{i=1}^m f(G(Z_i))$  as loss function;
24   end
25   Testing step: while (test samples) do
26     Generator.fit(samples, epochs, batch);
27     Apply  $\Rightarrow$  Adam as optimizer;
28     Apply  $\Rightarrow \frac{G}{m} \sum_{i=1}^m f(G(Z_i))$  as loss function;
29   end
30   Initialization discriminator network;
31   input: epochs, batch, lr(learning rate);
32    $x_1, x_2, \dots, x_n \Rightarrow$  Ethereum real transactions;
33    $g_1, g_2, \dots, g_n \Rightarrow$  Adversarial transactions;
34   Discriminator =  $D_{DNN}(x_n, g_n)$ 
35   while (train samples) do
36     Discriminator. fit(samples, epochs, batch);
37     Apply  $\Rightarrow$  Adam as optimizer;
38     Apply  $\Rightarrow \frac{W}{m} \sum_{i=1}^m [f(x_i) - f(G(z_i))]$  as loss function;
39   end
40   Testing step;;
41   while (test samples) do
42     Discriminator. fit(samples, epochs, batch);
43     Apply  $\Rightarrow$  Adam as optimizer;
44     Apply  $\Rightarrow \frac{W}{m} \sum_{i=1}^m [f(x_i) - f(G(z_i))]$  as loss function;
45   end
46 end
47 Return generated transactions

```

delivery, exploitation, installation, command and control, and action on objectives. In the reconnaissance phase, the attacker tries to provide essential data for its purposes. Indeed, it collects and marshals information about valid transactions. This phase plays a vital role as long as fake transactions are produced based on this data. If this initial step is accomplished well, the possibility of bypassing the detection system will increase. In the weaponization step, the intruder makes an identical Ethereum transaction as a weapon and sends it to the Ethereum platform to deceive the cybersecurity systems and abuse it in the Delivery phase. The transaction places into a block to serve in the substitute phase, but before the finalization, it gets checked by cyber threat hunting models. So, the chain breaks, and the intruder cannot continue the process of disturbing a system. Indeed, it disrupts the following steps: Exploitation, Installation, Command & Control, and Action on the object.

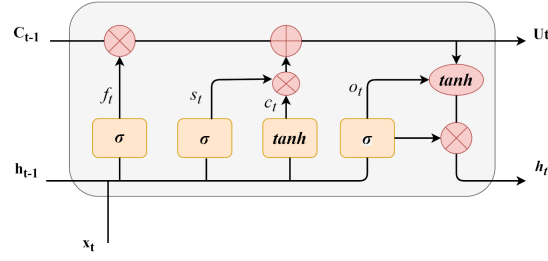


Fig. 3. The LSTM structure.

The main goal of the GAN-based threat hunting model is to disconnect one of the steps of CKC. Using this threat hunting model interrupts the 4th–7th steps, including Exploitation, Installation, Command & Control, and Action on an object in the chain, and prevents adversarial attacks. This means that the possibility of abusing the system will be eliminated before executing fake transactions. Due to the high-ranked similarity between fake and real transactions, the kill chain progresses without interruption, and the hacker can abuse the system. However, the proposed treat hunting model can eradicate this problem and break the chain from the exploitation phase.

We adopt bi_directional LSTM to create a cyber threat hunting model that quickly detects adversarial attacks in the Ethereum blockchain. At first, the data are preprocessed. In the preprocessing, we adopt the feature_selection method in ExtraTreesClassifier to choose the more useful feature for detecting attacks. ExtraTreesClassifier is a feature selection technique that uses multiple decision trees to classify the data. The reason behind using this method for feature selection is that Ethereum transactions have many features, and the ExtraTreesClassifier can reduce the feature space and model complexity. Moreover, the ExtraTreesClassifier uses a random split of all observations; the model does not overfit the data [47]. Then, we use Standard (St), which is shown in Equation (10), as a scaling method to transform data to a value between 0, and 1. In this equation, x represent the number of observation, m shows mean and d represent standard deviation.

$$St(x) = ((x - m))/d. \quad (10)$$

Then, the preprocessed data enter the bi-directional LSTM. The data is fed to the input layer, which has 60 neurons. After processing, the input layer transfers data to the hidden layers. We utilize four hidden layers with 80 neurons. Each hidden layer is equipped with a bi-directional LSTM. Bi-directional LSTM is an extended version of LSTM that uses two LSTMs. In the first round, an LSTM is applied to the input sequence in the forward direction. Then in the second round, the input sequence fed into LSTM in the backward direction. LSTM, as shown in Figure 3, has three gates: forget, input, and output. Each gate is responsible for controlling data transfer. The forget gate, which according to Equation (11), is responsible for preserving or removing the data. In this Equation, b and W represent the bias and weights of the LSTM cell.

$$F_t = \sigma(W_f[h(t-1)].W_f[x_t].b_f). \quad (11)$$

The input gate is responsible for deciding whether or not the new information adds to LSTM. The input gate consists of two layers. The sigmoid layer (S_i) decides which values need to be updated, and the tanh layer (C_t) that creates vectors from candidate data decides to add LSTM.

$$S_i = \sigma(W_i[h(t-1)].W_i[x_t].b_i), \quad (12)$$

$$C_t = \tanh(W_c[h(t-1)].W_c[x_t].b_c). \quad (13)$$

The combination of these layers provides an update in LSTM as follows:

$$U_t = F_t * C(t-1) + S_t * C_t. \quad (14)$$

The output gate is responsible for deciding what part of memory gives to the output:

$$H_t = \sigma(W_o[h(t-1)].W_o[x_t].b_o) \otimes \tanh(U_t). \quad (15)$$

The output of these hidden layers transfers to the output layer, which is responsible for representing the final decision about transactions status. For input and hidden layers, we applied ReLU, and for output, Sigmoid. This model is based on the **Binary Cross-Entropy (BCE)** cost function according to Equation (18). This equation includes $C1$ and $C2$, which indicate on the binary result. $t1 [0,1]$ and $S1$ are the ground truth, and the score for $C1$, and $t2=1-t1$ and $S2=1-S1$ are the ground truth and the score for $C2$.

$$CE = - \sum_{i=1}^{C'=2} t_i \log(s_i) = -t_1 \log(s_1) - (1-t_1) \log(1-s_1). \quad (16)$$

4 MODEL EVALUATION

We discuss the results of our proposed model. This section includes the experimental setup, dataset description, adversarial transaction generation results, and adversarial attack detection results. Also, we provide comparison results with other models.

4.1 Experimental Setup

The proposed model has been implemented in Python 3.7.4 in Windows 10. We used *Ctgan* and *Sdv* [48] libraries for generating tabular data. The GAN is trained for 500 epochs and 128 batch sizes to generate 14,250 fake transactions. The generator and discriminator learning rate is $2e-4$. The noise amount in the generator to generate fake transactions is 128. In the second phase of the preprocessing, we use the Extratree classifier to select the most effective features, as shown in Table 1. The model was trained with the 128 batch sizes for 20 epochs, and also we used 0.8 of the dataset for training and 0.2 for testing. Moreover, we adopted the optimal size of sliding windows for our model, which is 15. Both the stochastic gradient descent function and Adam optimizer were used to minimize errors. Generator and discriminator weight decay for the Adam Optimizer is $1e-6$.

4.2 Dataset

Choosing a suitable dataset to detect intrusion in Ethereum had several challenges. Initially, many datasets used to detect attacks in Ethereum transactions were self-generated. These datasets are of low quality and do not provide the correct form of transactions. These self-generated and low-quality datasets do not have an acceptable accuracy in detecting attacks and may raise false alarms in detecting fake transactions. On the other hand, many of the collected Ethereum datasets are unavailable to other researchers and are private due to copyright. Therefore, using an efficient dataset is one of the research requirements for detecting attacks in Ethereum. An efficient dataset must include realistic transactions, including correctly labeled transactions as normal and abnormal, sufficient data size, and all relevant transaction features such as value, gas, block number, and so on [49]. We assess our proposed model using a real Ethereum transaction dataset [50]. In addition to having the features for an efficient dataset, our chosen dataset is publicly available to researchers. Furthermore, the data in this dataset are preprocessed. Three critical methods, including cleaning, labeling, and reduction have been adopted in pre-processing. Preprocessing methods increase the dataset accuracy while using ML. Table 2 presents a qualitative comparison between the selected dataset and other existing ones.

This dataset contains 57,000 normal transactions and 14,250 abnormal transactions. These abnormal transactions include scamming and phishing attacks, which can be detected by detection systems straightforwardly. However, we removed these abnormal transactions as long as we focused on adversarial attacks only. In this way, fake transactions are produced from normal ones and have more potential to fool the detection system. Table 3 describes the statistics of normal and fraudulent transactions in the original and generated dataset.

Table 1. Features Importance

Features	Importance score
Receipt gas used	0.462
Gas	0.180
Gas price	0.105
Block number	0.082
Nonce	0.073
Receipt cumulative gas used	0.057
Value	0.037
Input	0.015
Transaction index	0.005

Table 2. Comparison between Datasets

Datasets	Labelled as normal & abnormal	Publicly available	Code availability	Preprocessing
[51]	✓	×	×	×
[52]	×	✓	✓	×
[53]	✓	×	×	×
[54]	✓	×	×	×
[55]	✓	×	×	×
[56]	✓	✓	×	✓
[57]	✓	×	×	×
Adopted dataset [50]	✓	✓	✓	✓

Table 3. The Statistics Summary of the Normal and Fraudulent Transactions in the Original and Generated Datasets

Datasets	Normal transactions	Fraudulent transactions
Original dataset	57,000	14,250
Generated dataset using GAN	57,000	1,000

Table 4 describes the features of each transaction. These selected features influence the detection system most, and the system gains a pattern to detect fraudulent transactions. However, since some features, such as hash, have a wide variety of values, and it is impossible to reach a transaction using its hash, they have not been considered.

In the feature selection process in the dataset, the most efficient features are selected to determine whether the transactions are genuine or counterfeit. We use an *Extratreeclassifier* for choosing important features. The purpose of feature selection is to increase relevance and decrease redundancy. Reducing the dataset size reduces the need for computational energy and training time and prevents the model from overfitting. *Extratreeclassifier* is made up of several random trees and belongs to the ensemble learning group [58]. K random features form each random tree from the feature set. Each tree uses the Gini Index GI to find the best feature for splitting the data. Equation (17) shows the GI where pi is the probability of an object being classified to a particular class. For choosing attributes according to the GI , these attributes are sorted in descending order, and the top k attributes are selected. Table 1 shows the degree of importance of the dataset properties to determine whether transactions are genuine or fake.

$$GI = 1 - \sum_{i=1}^n (pi)^2. \quad (17)$$

Table 4. Selected Feature Description

Feature	Description
Nonce	The nonce value related to blocks
Value	The value of each transaction
Gas	Quantity of gas, used by source
Gas price	The gas price, provided by source
Receipt gas used	Quantity of gas for each transaction
Receipt cumulative gas used	Quantity of gas for each transaction, in related block runtime
Block number	The number of each block
Transaction_index	The index of transaction in the block
input	The data sent together with the transaction

Table 5. GAN Evaluation Result with Different Criteria

Metrics	score
Logistic regression detection	0.9478
SVC detection	0.4992
Chi-Squared	1.0000
Inverted Kolmogorov–Smirnod	0.7584
Kullback–Leibler Divergence	0.9201

4.3 Adversarial Transaction Generation Results

We use a combination of metrics to evaluate the data generated by the model and consider their average as the model's overall score. Table 5 shows the model score for each evaluation metric. The model's overall score obtained from the average scores in Table 5 is 82.51%. In other words, this value shows the similarity rate of generated transactions and real transactions.

Figure 4 shows the distribution of generated values relative to the real data in the selected properties. In this figure, the x-axis shows the range of each feature of generated transactions, and the y-axis shows their density in that range. As we can see, the density and distribution of fake and real transactions are too close and overlap in most cases. The result of the GAN model is outstanding and with high accuracy when the distribution and density of generated fake transactions are close to real ones. In this case, it is difficult to distinguish fake data from real ones. Therefore, Figure 4 demonstrates that our model generated highly accurate adversarial transactions. Also, the mean and **standard deviation (STD)** of the real and generated data are presented in Figure 5. The mean and standard deviation of the generated data is consistently consistent with the real data.

4.4 Adversarial Attack Detection Result

The dataset consists of 14,250 fake transactions generated in the first phase, and 57,000 real transactions are the neural network input. The model evaluation result shows that precision, accuracy, F_measure, and recall are 0.9781, 0.9998, 0.9806, and 0.9832. These precision, accuracy, F_measure, and recall values are obtained in the order form (18), (19), (20), and (21).

$$Precision = \frac{TP}{TP + FP}, \quad (18)$$

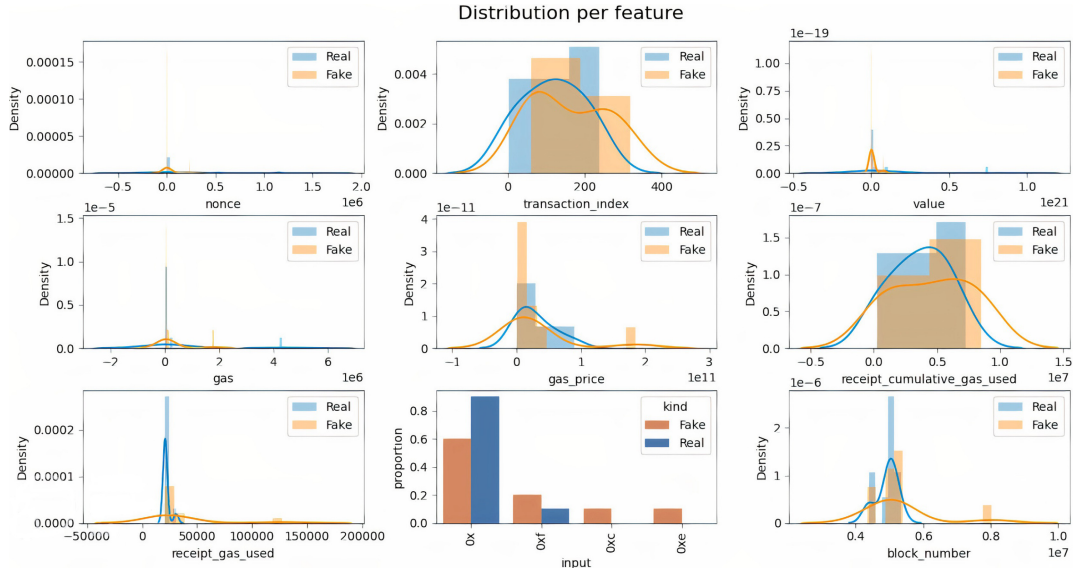


Fig. 4. Generated data distribution.

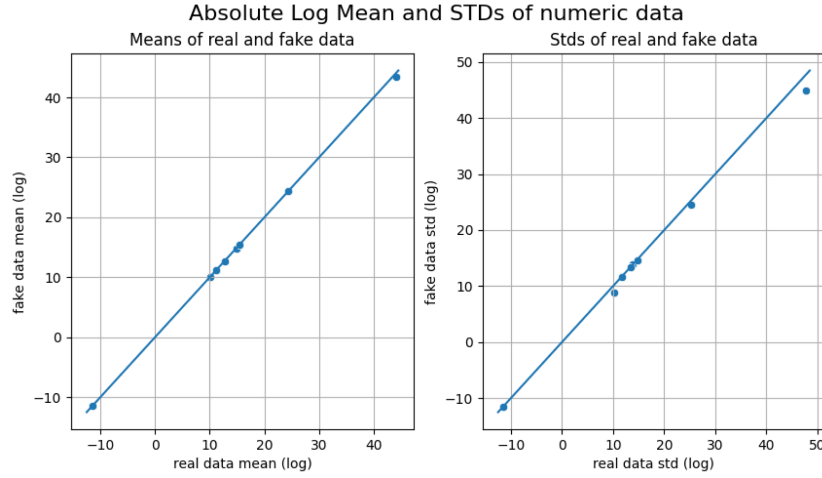


Fig. 5. Mean and STD of the generated data vs. real data.

$$Accuracy = \frac{TP + TN}{Total}, \quad (19)$$

$$F - measure = \frac{2 \times (Precision \times Recall)}{Precision + Recall}, \quad (20)$$

$$Recall = \frac{TP}{TP + FN}. \quad (21)$$

In addition, Figure 6 shows model accuracy for train and test. The model's accuracy for the training set was initially 96.99%, which reached 100% after 20 iterations. Also, the test set accuracy was 99.40% at the beginning and jumped to 99.98% at the 20th epoch.

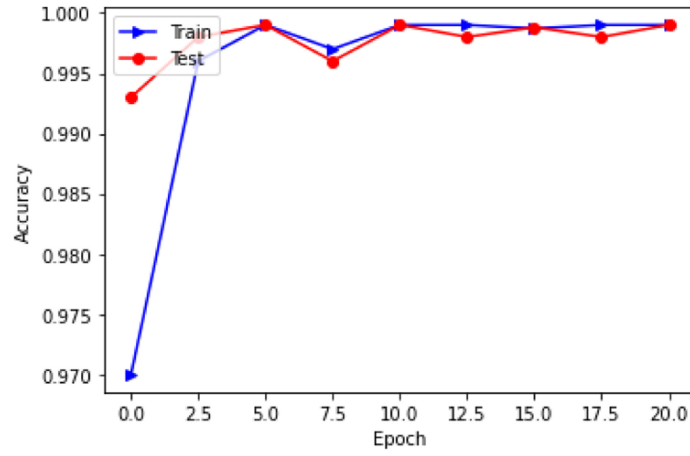


Fig. 6. Adversarial attack detection accuracy.

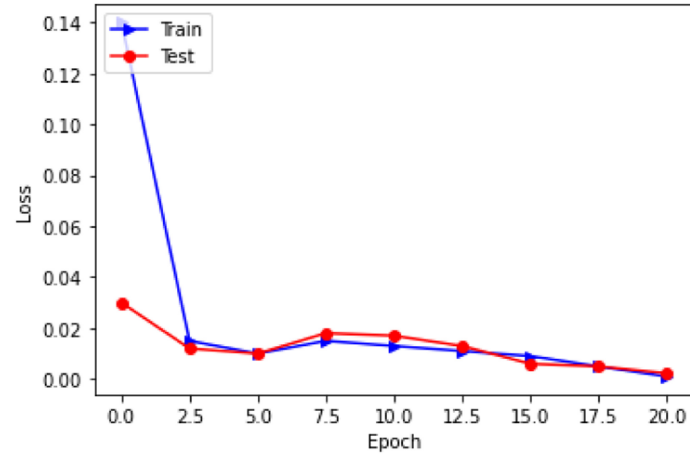


Fig. 7. Adversarial attack detection loss.

According to Figure 7, the loss function for training has decreased from 0.1349 to 5.8731×10^{-4} . Also, for the test set, the loss function has fallen from 0.2949 to 0.0023.

We also evaluate our model in terms of attack classification. Hence, we first consider our model for binary classification in a way that distinguishes adversarial transactions from normal ones. We then evaluate its effectiveness when the dataset contains scamming, phishing, and adversarial attacks. We calculated the result of binary classification according to Equations (18)–(21). The precision, accuracy, F-measure, and recall based on the confusion matrix (as shown in Figure 8), are 0.9871, 0.9752, 0.9877, and 0.9885.

For multi-classification, the abnormal transactions in the original database (which includes scamming and phishing attacks) and the generated transactions have been added to the database. Therefore, the datasets evaluated at this stage include 57,000 normal transactions, 14,250 abnormal transactions, including phishing and scamming attacks, and 1,000 fraudulent transactions. The model evaluation result shows that precision, accuracy, F-measure, and recall are 0.7696, 0.7662, 0.8443, and 0.9352, calculated based on the confusion matrix in Figure 9.

		Actual Label	
		1	0
Predicted Label	1	11252	157
	0	130	61

Fig. 8. The binary classification confusion matrix.

		Actual Label	
		1	0
Predicted Label	1	9053	2709
	0	627	2061

Fig. 9. The attack classification confusion matrix.

4.5 Comparison with Other Models

We have compared our model with several ML models. We applied **Naïve Bayes (NB)**, **Support Vector Machine (SVM)**, **Random Forest (RF)**, **K-Nearest Neighbor (KNN)**, **Multi-Level Perceptron (MLP)**, and **Decision Tree (DT)** models for comparison. As shown in Figure 10, the proposed model offers the best performance compared to the other models. The second rank belongs to the DT, which has an accuracy of 99.88%. The RF has an accuracy close to the DT, which is 99.85%. The MLP with three hidden layers, trained in the range of 50 rounds, has the next rank equal to 97.45%. Next is the KNN algorithm. This algorithm can detect adversarial transactions with 96.41% accuracy. Finally, SVM and NB have the lowest performance in detecting adversarial attacks. SVM can detect attacks with 96.21% accuracy, and NB can detect attacks with 91.06% accuracy.

Suppose we consider adversarial learning issues that provide concern in the ML area. In that case, our proposed approach fulfills better cyber threat hunting than the different mentioned approaches since it reduces the dimensions, extracts important features, and uses a bi-directional LSTM and conditional GAN to produce adversarial. Indeed, in comparison to other models, the proposed model can derive the unknown algorithm and adversary condition in normal operating conditions of the system by supervised and unsupervised training levels, so it will be more acceptable when applying real-world applications. The thing that needs to be highlighted here is that the proposed GAN-based threat hunting model in the Ethereum blockchain is more robust than other models, particularly regarding adversarial attacks, since it has been trained with the GAN approach.

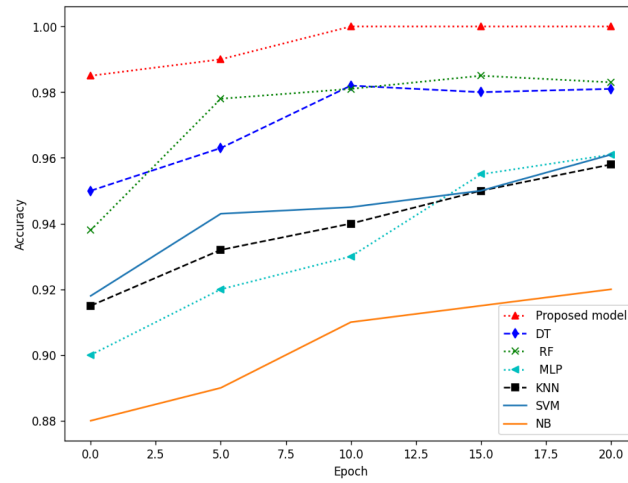


Fig. 10. Other models comparison for adversarial attack detection.

4.6 Implications

This article provided a practical model for threat hunting for the ML-based IDS in the Ethereum blockchain networks. The proposed model is helpful for both the industry and academic fields. Recently, Ethereum-run Dapps, such as Defi, have versatile applications in different domains in the industry. Due to their popularity and high usage, Dapps have become a new target for hacker attacks. In this way, our proposed model can help deal with the vulnerabilities and security problems of these Dapps and lessen the users' concerns. In addition, unknown and zero-day attacks are popular among hackers today, and even these vulnerabilities have been traded on the black market. Because zero-day attacks have targeted many prominent companies, these companies pay researchers who discover vulnerabilities. Since Dapps have complex environments, it is hard for researchers to find vulnerabilities manually; thus, approaches like ours could be helpful. Furthermore, this article's results show that the GAN-based threat hunting model can improve the IDS security and reliability by providing an extra security layer for detecting unknown attacks in Dapps. Moreover, for academic purposes, the results of this article can also help researchers accelerate the process of improving the efficiency and security of IDSs in the blockchain network by considering unknown and various attacks to reduce significant cost failures.

5 FUTURE WORKS

Several avenues for future research can be built on top of the current work. First, due to the blockchain's popularity, the number of transactions created in blockchain-based systems is increasing. The increasing popularity of blockchain makes it a perfect target for hackers. Therefore, there is a need for anomaly detection and threat hunting systems that can detect adversarial attacks in a short time with high accuracy. However, despite advantages, GAN-based cyber threat hunting in the blockchain is time and energy-consuming [22]. Furthermore, designing lightweight GAN models for threat hunting in blockchain should be considered in future works. Moreover, our proposed model's performance depends on training the GAN model correctly. So, to provide more accurate threat hunting models, the performance of GAN models should improve in future works to be more robust in real-world applications. In addition, in the future, to increase the accuracy of the GAN structure in generating similar transactions, we plan to experiment with other advanced architectures. Another critical challenge that should be considered in the future is the importance of data protection in the GAN model. If hackers access the training set and add malicious data to it, the GAN will be trained inappropriately and affect the whole threat hunting model. Therefore, data privacy-preserving should be carefully considered in further works.

6 CONCLUSION

Blockchain-based applications have become a new target for hackers because of the wide range of heavily indexed projects on decentralized finance in Web3. In the current situation, cyberattacks are increasing and more accessible, becoming much more attractive to hackers. Although one of the promising and paramount methods to detect attacks in blockchain networks is ML, it is not alone immune to hostile attacks. Adversarial attacks are attacks that can fool ML methods and abuse many systems outside the blockchain and will undoubtedly reach this domain sooner than later. To address the issues with adversarial threat hunting through detecting cyber attacks in the Ethereum blockchain, we proposed a two-phase ML model using GAN and DNN. The first phase was responsible for creating adversarial transactions on the Ethereum blockchain using GAN, and the second phase was responsible for detecting adversarial attacks using bi-directional LSTM. We evaluated our proposed model using Ethereum's transaction dataset, and 99.98% accuracy was achieved.

REFERENCES

- [1] Ahmed Afif Monrat, Olov Schelén, and Karl Andersson. 2019. A survey of blockchain from the perspectives of applications, challenges, and opportunities. *IEEE Access* 7, 7 (2019), 117134–117151.
- [2] Mohamed Amine Ferrag and Lei Shu. 2021. The performance evaluation of blockchain-based security and privacy systems for the Internet of Things: A tutorial. *IEEE Internet of Things Journal* 8, 24 (2021), 17236–17260.
- [3] Gaby G. Dagher, Praneeth Babu Marella, Matea Milojkovic, and Jordan Mohler. 2018. Broncovote: Secure voting system using ethereum's blockchain. (2018).
- [4] Jia Qu. 2022. Blockchain in medical informatics. *Journal of Industrial Information Integration* 25, 25 (2022), 100258.
- [5] Thomas Bocek, Bruno B. Rodrigues, Tim Strasser, and Burkhard Stiller. 2017. Blockchains everywhere-a use-case of blockchains in the pharma supply-chain. In *Proceedings of the 2017 IFIP/IEEE Symposium on Integrated Network and Service Management*. IEEE, 772–777.
- [6] Emanuel Ferreira Jesus, Vanessa R. L. Chicarino, Célio V. N. De Albuquerque, and Antônio A. de A. Rocha. 2018. A survey of how to use blockchain to secure internet of things and the stalker attack. *Security and Communication Networks* 2018 (2018), 27 Pages.
- [7] Liya Su, Xinyue Shen, Xiangyu Du, Xiaojing Liao, XiaoFeng Wang, Luyi Xing, and Baoxu Liu. 2021. Evil under the sun: Understanding and discovering attacks on Ethereum decentralized applications. In *Proceedings of the 30th USENIX Security Symposium*. 1307–1324.
- [8] Simon Joseph Aquilina, Fran Casino, Mark Vella, Joshua Ellul, and Constantinos Patsakis. 2021. EtherClue: Digital investigation of attacks on Ethereum smart contracts. *Blockchain: Research and Applications* 2, 4 (2021), 100028.
- [9] Iuon-Chang Lin and Tzu-Chun Liao. 2017. A survey of blockchain security issues and challenges. *International Journal of Network Security* 19, 5 (2017), 653–659.
- [10] Amritraj Singh, Reza M. Parizi, Qi Zhang, Kim-Kwang Raymond Choo, and Ali Dehghantanha. 2020. Blockchain smart contracts formalization: Approaches and challenges to address vulnerabilities. *Computers and Security* 88, 88 (2020), 101654. DOI: <http://dx.doi.org/10.1016/j.cose.2019.101654>
- [11] Bin Xia, Junjie Yin, Jian Xu, and Yun Li. 2019. LogGAN: A sequence-based generative adversarial network for anomaly detection based on system logs. In *Proceedings of the International Conference on Science of Cyber Security*. Springer, 61–76.
- [12] Zhi Chen, Jiang Duan, Li Kang, and Guoping Qiu. 2021. Supervised anomaly detection via conditional generative adversarial network and ensemble active learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2021).
- [13] Zengguang Liu and Xiaochun Yin. 2021. LSTM-CGAN: Towards generating low-rate DDoS adversarial samples for blockchain-based wireless network detection models. *IEEE Access* 9, 9 (2021), 22616–22625.
- [14] Rachit Agarwal, Tanmay Thapliyal, and Sandeep K. Shukla. 2021. Detecting malicious accounts showing adversarial behavior in permissionless blockchains. arXiv:2101.11915. Retrieved from <https://arxiv.org/abs/2101.11915>.
- [15] Muhammad Saad, My T. Thai, and Aziz Mohaisen. 2018. POSTER: Deterring ddos attacks on blockchain-based cryptocurrencies through mempool optimization. In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security*. 809–811.
- [16] Ben Charoenwong and Mario Bernardi. 2021. A decade of cryptocurrency 'hacks': 2011–2021. Available at SSRN 3944435.
- [17] Abbas Yazdinejad, Hamed HaddadPajouh, Ali Dehghantanha, Reza M. Parizi, Gautam Srivastava, and Mu-Yen Chen. 2020. Cryptocurrency malware hunting: A deep recurrent neural network approach. *Applied Soft Computing* 96, 96 (2020), 106630.
- [18] Benjamin Bowman and H. Howie Huang. 2021. Towards next-generation cybersecurity with graph AI. *ACM SIGOPS Operating Systems Review* 55, 1 (2021), 61–67.
- [19] Mohamed Amine Ferrag, Leandros Maglaras, Sotiris Moschogiannis, and Helge Janicke. 2020. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications* 50, 50 (2020), 102419.
- [20] Tommaso Zoppi and Andrea Ceccarelli. 2021. Prepare for trouble and make it double! Supervised–Unsupervised stacking for anomaly-based intrusion detection. *Journal of Network and Computer Applications* 189, 189 (2021), 103106.
- [21] Ngoc-Trung Tran, Viet-Hung Tran, Ngoc-Bao Nguyen, Trung-Kien Nguyen, and Ngai-Man Cheung. 2021. On data augmentation for GAN training. *IEEE Transactions on Image Processing* 30, 21 (2021), 1882–1897.

- [22] Zhipeng Cai, Zuobin Xiong, Honghui Xu, Peng Wang, Wei Li, and Yi Pan. 2021. Generative adversarial networks: A survey toward private and secure applications. *ACM Computing Surveys* 54, 6 (2021), 1–38.
- [23] Naila Mukhtar, Lejla Batina, Stjepan Picek, and Yinan Kong. 2022. Fake it till you make it: Data augmentation using generative adversarial networks for all the crypto you need on small devices. In *Proceedings of the Cryptographers' Track at the RSA Conference*. Springer, 297–321.
- [24] Abbas Yazdinejad, Behrouz Zolfaghari, Ali Dehghantanha, Hadis Karimipour, Gautam Srivastava, and Reza M. Parizi. 2022. Accurate threat hunting in industrial internet of things edge devices. *Digital Communications and Networks* 54 (2022). DOI : <http://dx.doi.org/10.1016/j.dcan.2022.09.010>
- [25] Wenbo Zheng, Kunfeng Wang, and Fei-Yue Wang. 2020. Gan-based key secret-sharing scheme in blockchain. *IEEE Transactions on Cybernetics* 51, 1 (2020), 393–404.
- [26] Jianxiong Lai, Xiuli Huang, Xianzhou Gao, Chang Xia, and Jingyu Hua. 2022. GAN-based information leakage attack detection in federated learning. *Security and Communication Networks* 2022 (2022).
- [27] Abbas Yazdinejad, Ali Dehghantanha, Reza M. Parizi, Mohammad Hammoudeh, Hadis Karimipour, and Gautam Srivastava. 2022. Block hunter: Federated learning for cyber threat hunting in blockchain-based IIoT networks. *IEEE Transactions on Industrial Informatics* 18, 11 (2022), 8356–8366. DOI : <http://dx.doi.org/10.1109/TII.2022.3168011>
- [28] Abbas Yazdinejadna, Reza M. Parizi, Ali Dehghantanha, and Hadis Karimipour. 2021. Federated learning for drone authentication. *Ad Hoc Networks* 120, 120 (2021), 102574.
- [29] Qingyu Guo, Zhao Li, Bo An, Pengrui Hui, Jiaming Huang, Long Zhang, and Mengchen Zhao. 2019. Securing the deep fraud detector in large-scale e-commerce platform via adversarial machine learning approach. In *Proceedings of the World Wide Web Conference*. 616–626.
- [30] Ivan Furosov, Matvey Morozov, Nina Kaplounkhaya, Elizaveta Kovtun, Rodrigo Rivera-Castro, Gleb Gusev, Dmitry Babaev, Ivan Kireev, Alexey Zaytsev, and Evgeny Burnaev. 2021. Adversarial attacks on deep models for financial transaction records. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2868–2878.
- [31] Hung Ba. 2019. Improving detection of credit card fraudulent transactions using generative adversarial networks. arXiv:1907.03355. Retrieved from <https://arxiv.org/abs/1907.03355>.
- [32] Eunbi Seo, Hyun Min Song, and Huy Kang Kim. 2018. Gids: Gan based intrusion detection system for in-vehicle network. In *Proceedings of the 2018 16th Annual Conference on Privacy, Security and Trust*. IEEE, 1–6.
- [33] Dan Li, Dacheng Chen, Jonathan Goh, and See-kiong Ng. 2018. Anomaly detection with generative adversarial networks for multivariate time series. arXiv:1809.04758. Retrieved from <https://arxiv.org/abs/1809.04758>.
- [34] Phuc Cuong Ngo, Amadeus Aristo Winarto, Connie Khor Li Kou, Sojeong Park, Farhan Akram, and Hwee Kuan Lee. 2019. Fence GAN: Towards better anomaly detection. In *Proceedings of the 2019 IEEE 31st International Conference on Tools with Artificial Intelligence*. IEEE, 141–148.
- [35] Auwal Sani Ilyasu and Huifang Deng. 2022. N-GAN: A novel anomaly-based network intrusion detection with generative adversarial networks. *International Journal of Information Technology* 14, 14 (2022), 1–11.
- [36] G. Sugitha, A. Solairaj, and J. Suresh. 2022. Block chain fostered cycle-consistent generative adversarial network framework espoused intrusion detection for protecting IoT network. *Transactions on Emerging Telecommunications Technologies* 33, 11 (2022), e4578.
- [37] Francesco Scicchitano, Angelica Liguori, Massimo Guarascio, Ettore Ritacco, and Giuseppe Manco. 2020. A deep learning approach for detecting security attacks on blockchain. In *Proceedings of the ITASEC*. 212–222.
- [38] Dule Shu, Nandi O. Leslie, Charles A. Kamhoua, and Conrad S. Tucker. 2020. Generative adversarial attacks against intrusion detection systems using active learning. In *Proceedings of the 2nd ACM Workshop on Wireless Security and Machine Learning*. 1–6.
- [39] Alexey Kurakin, Ian J. Goodfellow, and Samy Bengio. 2018. Adversarial examples in the physical world. In *Proceedings of the Artificial Intelligence Safety and Security*. Chapman and Hall/CRC, 99–112.
- [40] Houssam Zenati, Chuan Sheng Foo, Bruno Lecouat, Gaurav Manek, and Vijay Ramaseshan Chandrasekhar. 2018. Efficient gan-based anomaly detection. arXiv:1802.06222. Retrieved from <https://arxiv.org/abs/1802.06222>.
- [41] David Liu and Nathan Hu. 2020. *GAN-Based Image Data Augmentation*. Stanford University.
- [42] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. 2019. Modeling tabular data using conditional gan. *Advances in Neural Information Processing Systems* 32 (2019).
- [43] Xuan Xia, Xizhou Pan, Nan Li, Xing He, Lin Ma, Xiaoguang Zhang, and Ning Ding. 2022. GAN-based anomaly detection: A review. *Neurocomputing* 493, 493 (2022), 497–535.
- [44] Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical reparameterization with gumbel-softmax. arXiv:1611.01144. Retrieved from <https://arxiv.org/abs/1611.01144>.
- [45] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. 2017. Improved training of wasserstein gans. *Advances in Neural Information Processing Systems* 30 (2017).
- [46] Pooneh Nikkhah Bahrami, Ali Dehghantanha, Tooska Dargahi, Reza M. Parizi, Kim-Kwang Raymond Choo, and Hamid H. S. Javadi. 2019. Cyber kill chain-based taxonomy of advanced persistent threat actors: Analogy of tactics, techniques, and procedures. *Journal of Information Processing Systems* 15, 4 (2019), 865–889.

- [47] Monika Arya, Hanumat Sastry, Anand Motwani, Sunil Kumar, and Atef Zaguia. 2021. A novel extra tree ensemble optimized DL framework (ETEODL) for early detection of diabetes. *Frontiers in Public Health* 9, 9 (2021).
- [48] Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. 2016. The synthetic data vault. In *Proceedings of the 2016 IEEE International Conference on Data Science and Advanced Analytics*. IEEE, 399–410.
- [49] Anna Sperotto, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras, and Burkhard Stiller. 2010. An overview of IP flow-based intrusion detection. *IEEE Communications Surveys and Tutorials* 12, 3 (2010), 343–356.
- [50] Al-E'mari Salam, Mohammed Anbar, Yousef Sanjalawe, and Selvakumar Manickam. 2020. A labeled transactions-based dataset on the ethereum network. In *Proceedings of the International Conference on Advances in Cyber Security*. Springer, 61–79.
- [51] Steven Farrugia, Joshua Ellul, and George Azzopardi. 2020. Detection of illicit accounts over the Ethereum blockchain. *Expert Systems with Applications* 150, 150 (2020), 113318.
- [52] Shlomi Linoy, Natalia Stakhanova, and Alina Matyukhina. 2019. Exploring Ethereum's blockchain anonymity using smart contract code attribution. In *Proceedings of the 2019 15th International Conference on Network and Service Management*. IEEE, 1–9.
- [53] Saikumar Nair and Jignasa V. Gohel. 2020. A review on contemporary hole transport materials for perovskite solar cells. *Nanotechnology for Energy and Environmental Engineering* (2020), 145–168.
- [54] Ross Phillips and Heidi Wilder. 2020. Tracing cryptocurrency scams: Clustering replicated advance-fee and phishing websites. In *Proceedings of the 2020 IEEE International Conference on Blockchain and Cryptocurrency*. IEEE, 1–8.
- [55] Blaž Podgorelec, Muhamed Turkanović, and Sašo Karakatič. 2020. A machine learning-based method for automated blockchain transaction signing including personalized anomaly detection. *Sensors* 20, 1 (2020), 147.
- [56] Weili Chen, Zibin Zheng, Edith C.-H. Ngai, Peilin Zheng, and Yuren Zhou. 2019. Exploiting blockchain data to detect smart ponzi schemes on ethereum. *IEEE Access* 7, 7 (2019), 37575–37586.
- [57] Jiajing Wu, Qi Yuan, Dan Lin, Wei You, Weili Chen, Chuan Chen, and Zibin Zheng. 2020. Who are the phishers? Phishing scam detection on ethereum via network embedding. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 52, 2 (2020), 1156–1166.
- [58] Aakanksha Sharaff and Harshil Gupta. 2019. Extra-tree classifier with metaheuristics approach for e-mail classification. In *Proceedings of the Advances in Computer Communication and Computational Sciences*. Springer, 189–197.

Received 10 December 2021; revised 30 September 2022; accepted 23 October 2022